

Session 02 - Primitive DataTypes, Tuple, Arrays, Type Alias, Objects

Naveen AutomationLabs | 2026 Playwright Batch G1

1. Variable Declaration with Types

In TypeScript, you declare the type of a variable using a colon after the variable name. Syntax: `let varName: type = value;`

```
let num: number = 100;
console.log(num);
console.log(typeof num); // 'number'

let username: string = 'admin';
console.log(username);

let isActive: boolean = true;
isActive = false;
console.log(isActive);
```

2. Special Types: null, undefined, bigint

```
let user: null = null;
let x: undefined = undefined;

let distance: bigint = 100n;
console.log(distance + 10n); // 110n
```

bigint is for very large numbers. Use the 'n' suffix to create bigint values. You cannot mix bigint with regular numbers.

3. any and unknown

The 'any' type turns off type checking. The variable can hold any value. Use it sparingly.

```
let testValue: any;
testValue = 100;
testValue = 'testing';
testValue = true;
console.log(testValue); // true
```

The 'unknown' type is a safer version of 'any'. You must check the type before using it.

```
let value: unknown = "hello";
console.log(value);
console.log(typeof value); // 'string'
```

Prefer 'unknown' over 'any' when you are not sure about the type. It forces you to do type checking before using the value.

4. void and never

void means the function does not return anything. never means the function will never complete normally -- it always throws an error or runs forever.

```
// void: no return value
function testing(): void {
  console.log('hello testing');
}
testing();

function getMarks(stName: string): void {
  console.log(stName, 100);
  console.log(stName.length);
}
getMarks('Tom');

// never: always throws an error or infinite loop
function throwElementError(locator: string): never {
  throw new Error(locator + " is not found error...");
}

function throwEnvNotFoundError(envName: string): never {
  throw new Error("INVALID ENV:" + envName);
}
```

5. Union Types

A union type allows a variable to hold more than one type. Use the | (pipe) symbol.

```
let id: string | number;
id = "abc1245"; // valid
id = 12345; // also valid
console.log(id);
```

6. Arrays

```
// Number array:
let marks: number[] = [10, 20, 30, 40];
console.log(marks);
console.log(marks.length);

// String array:
let devices: string[] = ['macbook pro', 'airtel sim', 'iphone 17', 'imac'];

// Mixed array using generics:
let names: Array<string | number> = ["tom", "kunal", "pooja", 100];
let salary: Array<number> = [100, 200, 300];
```

7. Tuple

A tuple is a fixed-length array where each position has a specific type. Think of it as a static, structured array.

```
let myuser: [string, number] = ['somika', 34.44];
console.log(myuser);

let person: [string, string, number, boolean] = ['tom', 'autom', 12.33, true];
```

Unlike regular arrays, tuples enforce both the type and the position of each element.

8. Type Alias (Custom Types)

Use the 'type' keyword to create a custom type. This is useful for defining object shapes and reusable union types.

```
// Custom type for objects:
type userType = {
  readonly name: string,
  salary: number,
  isActive: boolean,
  city: string
};

let newUser: userType = {
  name: 'naveen',
  salary: 12.33,
  isActive: true,
  city: 'Bangalore'
};

console.log(newUser);
newUser.salary = 15.55; // allowed
// newUser.name = 'abc'; // Error: readonly property

// Custom type for union:
type orderIdType = string | number;
let orderId: orderIdType = 1234;
orderId = '12345'; // also valid

type statusCodeType = string | number;
let okStatusCode: statusCodeType = 200;
okStatusCode = '200 OK';
```

readonly means the property can be set once during creation but cannot be changed later.